Project Reflection: FreedomPath PH

Czarina Oyales

I built FreedomPath PH because "how much should I actually be saving or investing" is a question I've bounced around in my own head more than once, and every time I looked for an answer online I got one of two things: a generic retirement calculator built for a US audience that doesn't know what Pag-IBIG or SSS even are, or a blog post comparing bank rates that says nothing about whether that rate is actually enough for your goal. Nothing tied those two things together: how much you need, where it should go, and whether your current habits are actually closing that gap. That's the gap I wanted this app to fill, kept small enough that I could actually finish it.

What I actually built

The app asks someone a short set of questions (age, income, current savings, their financial goal, and a few questions about how they feel about risk) and turns that into a personalized plan: how much they need to save each month, a rough estimate of their future SSS pension, and where their money should go across an HYSA (a high-yield savings account for their emergency fund), MP2, and a UITF fund matched to their risk comfort. Anyone can create a free account to save their plan and come back later to update it.

The AI does genuine work here, not just decoration on a calculator. It reads someone's risk answers and classifies their risk tolerance, then writes the personalized, plain-language narrative that explains their plan and recommends the specific HYSA/MP2/UITF split based on that profile. What I did put a hard line around is the actual arithmetic: the SSS pension formula and the required-savings math are handled by plain, dependable code, verified against known worked examples, and the AI is only allowed to cite rates that exist in my own reference data, never invent one. That's a deliberate choice, not a sign the AI isn't doing much: the personalization and the recommendation are the actual product.

How much Claude helped

I want to be upfront: Claude was involved in basically every step, not just the coding. I used it inside Cursor while writing the app, and talked through decisions with it separately, from setting up the project structure, to the forms and screens, to the AI recommendation piece, to accounts and login. It wasn't just autocomplete. It was more like having someone to think out loud with.

Where it helped the most was bugs. Whenever something behaved strangely, I'd describe what I was seeing and Claude would help me trace what was actually going on and fix it. The trickiest example: if someone builds a plan without an account and then decides to save it, they get sent to sign up or log in first, and I didn't want them to come back to a blank form after typing in all their numbers. Claude actually clicked through the real flow step by step (sign out, rebuild a plan, save it, sign back in, check what shows up) until it was confident it worked, rather than assuming the code looked right.

What was harder than I expected

Staying small was the recurring fight. My early validation work had already warned me that trying to do too much was the biggest risk for a project like this, and I still had to keep talking myself out of adding "just one more feature" while building. Sticking to the HYSA, MP2, one UITF tier, and the SSS estimate, and leaving things like individual stock picks or notifications for later, was the right call, but it took real effort to keep choosing it. I also had to sit with something less technical: figuring out, in plain terms, where "general educational information" ends and "advice that legally requires a license" begins. I'm not a lawyer, and this isn't legal advice, but I did enough digging to land on a reasonable, honest position for a free student project (general language, no named funds, no charging money) and wrote that reasoning down instead of just hoping it would be fine.

Biggest takeaway

If I had to point to one thing, it's that the work I did before writing any code mattered as much as the code itself. Spending real time validating the idea and writing an actual PRD, instead of jumping straight into building, meant I already knew what to say no to before I was tempted to add it in the moment. Skipping that step and figuring out scope as I went would have been a much worse way to build this.

The second thing I'd tell myself next time is to trust building iteratively even more than I did. Making small, incremental changes and actually testing the user experience after each one made it so much easier to catch problems early and decide whether something needed to change, instead of discovering an issue after everything was already tangled together. That habit, paired with a genuinely minimal MVP, is what let me finish a working, deployed app instead of a half-built version of something bigger.